

# Sertifikalar ve Kriptografik Yöntemler

CEN429 Güvenli Programlama — Hafta 10

Dr. Öğr. Üyesi Uğur CORUH · 20.11.2026

## Bugünün planı (3 saat)

| Saat | Bölüm | Konu                                                                                      |
|------|-------|-------------------------------------------------------------------------------------------|
| 1    | 1–3   | Algoritma/anahtar seçimi · blok şifre kipleri/dolgu · MAC/HMAC                            |
| 2    | 4–6   | RSA/ECC (OAEP/PSS) · dijital imza · Diffie–Hellman                                        |
| 3    | 7–13  | PKI · X.509 · OpenSSL ile zincir kurma · CRL/OCSP · HSM/PKCS#11 · kuantum sonrası · proje |

**Öğrenme çıktıları (ÖÇ.2 / ÖÇ.4):** doğru algoritma, kip, dolgu ve anahtar uzunluğu seçmek · imza ve anahtar

değişiminin tuzaklarını görmek · bir sertifika zincirini doğru doğrulamak

Kriptoda hatalar genelde algoritmada değil, **kullanımdadır**: yanlış kip, yanlış dolgu, doğrulanmayan imza, denetlenmeyen zincir.

- **Simetrik ve asimetrik şifreleme** — simetrik tek paylaşılan anahtarla hızlı çalışır; asimetrik açık/özel anahtar çiftiyle anahtar dağıtımını kolaylaştırır ama yavaştır (**Hafta 3**)
- **AEAD** — şifreleme ve bütünlüğü tek çağrıda birlikte veren kimlik doğrulamalı şifreleme (**Hafta 3**)
- **Özet, MAC ve dijital imza** — özet tek yönlü parmak izidir; MAC paylaşılan anahtarla, imza açık/özel anahtar çiftiyle mesajın değişmediğini kanıtlar (**Hafta 3**)
- **İleri gizlilik (forward secrecy)** — oturum anahtarının kullanımdan sonra silinmesiyle, bugünkü anahtar sızsa bile geçmiş oturumların çözülmemesi (**Hafta 3**)
- **TLS sertifika doğrulama** — istemcinin sunucu sertifikasını zincir, geçerlilik, kullanım amacı ve ad açısından denetlemesi (**Hafta 3**)

Bu hafta: asimetrik matematiği ve doğru kip/dolgu (Bölüm 2, 4) · HMAC iç yapısı ve imza tuzakları (Bölüm 3, 5) ·

## Bu haftanın kavramları

Her terim, gövdede ilk geçtiği yerde tanımlanır; burada yalnız **nerede** olduğunu işaretliyoruz.

| Kavram                    | Nerede   |
|---------------------------|----------|
| Blok şifre kipleri, dolgu | Bölüm 2  |
| MAC, HMAC                 | Bölüm 3  |
| RSA, eliptik eğri (ECC)   | Bölüm 4  |
| Dijital imza              | Bölüm 5  |
| Diffie–Hellman (DH)       | Bölüm 6  |
| PKI, CA                   | Bölüm 7  |
| Sertifika, X.509          | Bölüm 8  |
| OpenSSL ile zincir kurma  | Bölüm 9  |
| CRL, OCSP                 | Bölüm 10 |
| HSM, PKCS#11, SoftHSM     | Bölüm 11 |
| Kuantum sonrası (PQC)     | Bölüm 12 |

# 1. Algoritma ve anahtar seçimi

## Kısa tarihçe — anahtarlar, sertifikalar, PKI

- **1976–77** — Diffie–Hellman ve **RSA**: tanımadığınla güvenli konuşma
- **1988** — **X.509** sertifika biçimi; kimliği bir **CA imzası** taşır
- **1995** — ticari CA'lar ve **PKI**; ardından **CRL** ve **OCSP** (iptal)
- **2014** Heartbleed · **2015** Let's Encrypt · **2018** TLS 1.3 · **2022–24** **PQC** (Kyber/Dilithium)

Bugünkü kurallar (doğru kip/dolgu, zincir doğrulama, iptal) bu acı derslerden çıktı.

# Güncel, standart, iyi kullanılan

---

Üç kural:

- **Güncel:** kırılmamış algoritma (AES, SHA-256, Ed25519).
- **Standart:** kendi kriptonu yazma; denenmiş kütüphane.
- **İyi kullanılan:** doğru kip, dolgu, anahtar yönetimi.

## Kaçınılacaklar

---

- MD5, SHA-1 (özet için), DES/3DES, RC4.
- ECB kipi.
- Kendi "şifreleme" algoritman.
- Sabit/öngörülebilir IV veya anahtar.

# Anahtar uzunluğu

---

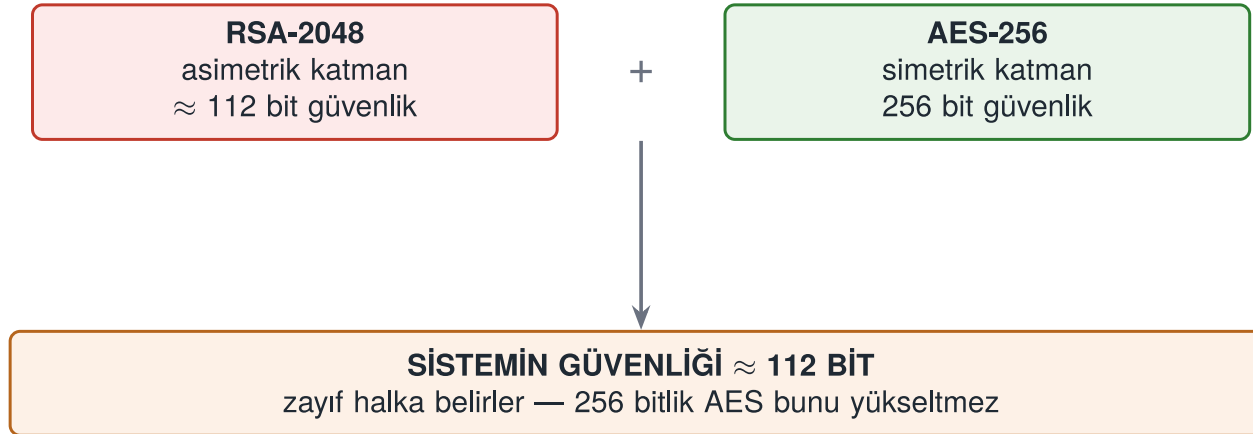
| Amaç     | Öneri (yaklaşık)              |
|----------|-------------------------------|
| Simetrik | AES-128 (yeter), AES-256      |
| RSA      | $\geq 2048$ , tercih 3072     |
| ECC      | 256-bit ( $\approx$ RSA-3072) |
| Özet     | SHA-256+                      |

---

# En zayıf halka kuralı

## En zayıf halka kuralı

sistemin gücü, en güçlü parçası kadar değildir



Katmanları ayrı ayrı değil, en düşük güvenlik seviyesine göre değerlendirin.

- Bileşenleri **dengeli** seç.
- 128 bit hedefliyorsan RSA-3072 ya da ECC.

## Hangi kip?

| İhtiyaç                      | Seçim                        |
|------------------------------|------------------------------|
| Gizlilik + bütünlük          | AES-GCM / ChaCha20-Poly1305  |
| Yalnız hız (donanım AES yok) | ChaCha20-Poly1305            |
| Asla                         | ECB                          |
| Eski sistem CBC              | + encrypt-then-MAC, tek hata |

## Hangi asimetrik?

---

| İhtiyaç           | Seçim                    |
|-------------------|--------------------------|
| İmza (modern)     | Ed25519                  |
| Anahtar anlaşması | X25519                   |
| RSA şifreleme     | RSA-OAEP ( $\geq 3072$ ) |
| RSA imza          | RSA-PSS                  |

---

## Karar kuralı

---

- Modern, standart, iyi kullanılan seç.
- AEAD öncelikli; ECC öncelikli.
- Kararı ve gerekçesini **S8**'e yaz.

## 2. Blok şifre kipleri ve dolgu

## ECB · asla

---

- **ECB:** her bloğu bağımsız şifreler.
- Aynı blok → aynı şifreli çıktı → **desen sızar.**
- Ünlü "ECB penguen" örneği.

ECB kullanma.

# ECB neden kötü — şema

## ECB neden asla kullanılmaz?

ünlü “ECB pengueni” bu yüzden görünür kalır

### ECB

her blok BAĞIMSIZ şifrlenir

aynı düz metin bloğu →  
aynı şifreli blok  
→ DESENLER görünür

### CBC / CTR / GCM

her blok öncekine ya da  
nonce'a bağlı

aynı metin → farklı şifreli  
→ desen sızmaz

Kip seçimi şifrelemenin kendisi kadar önemlidir; varsayılan tercih AEAD (GCM).

## CBC · dikkatli

---

- **CBC**: her blok bir öncekiyle zincirlenir; **IV** gerekir.
- IV **rastgele** ve tekrarsız olmalı.
- Tek başına **bütünlük vermez** → ayrı MAC şart.

# CBC + dolgu = risk

---

- CBC dolgu gerektirir.
- Yanlış dolgu işleme → **dolgu kâhini (padding oracle)** (birazdan).
- Bu yüzden modern tercih **AEAD**.

## PKCS#7 dolgusu · örnek

13 baytlık "MERHABA DUNYA" metnini AES-128-CBC ile şifreleyelim (K, IV yalnız gösterim için sabit):

```
printf 'MERHABA DUNYA' | openssl enc -aes-128-cbc -K "$K" -iv "$IV" -out cikti.bin  
xxd -p cikti.bin
```

Çıktı **16 bayt** (32 hex karakter) — girdi 13 bayttı, demek ki **3 bayt dolgu** eklendi.

## PKCS#7 dolgusu · çıktı

-nopad ile dolgunun içeriğini görelim (yalnız öğretici amaçla):

```
4d45 5248 4142 4120 4455 4e59 4103 0303  MERHABA DUNYA...
```

Son üç bayt **03 03 03**: eksik bayt sayısı ( $16 - 13 = 3$ ) kadar, o sayı değerinde bayt eklenmiş — tam olarak PKCS#7 kuralı. Normal `openssl enc -d` bu baytları otomatik okuyup atar.

# PKCS#7 · dolgu ve denetim — animasyon

data = 

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| S | e | c | r | e | t | M | e | s | s | a | g | e | 0 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

unpad — tüm kontroller geçti: orijinal 15 bayt geri verildi.

## GCM · modern tercih (AEAD)

---

- **AES-GCM:** gizlilik + bütünlük birlikte.
- Dolgu **yok**; bir **nonce** (tek kullanımlık sayı) gerekir.
- Nonce **asla** tekrar etmemeli (aynı anahtar altında).

## Nonce/IV kuralı

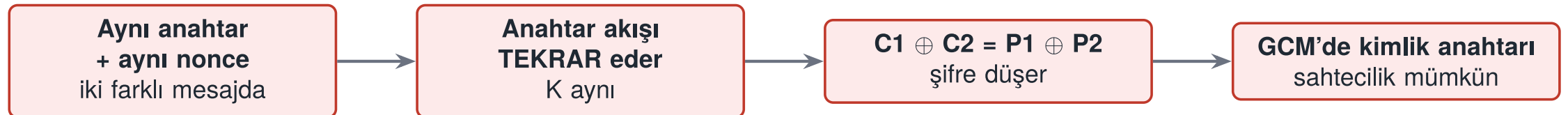
---

- IV/nonce **benzersiz** olmalı.
- GCM'de nonce tekrarı **felakettir** (anahtar/veri sızabilir).
- Sayaç ya da rastgele (yeterli uzunlukta) kullan.

# Nonce tekrarı — şema

## Nonce tekrarı neden felaket?

gizlilik VE bütünlük birlikte çöker



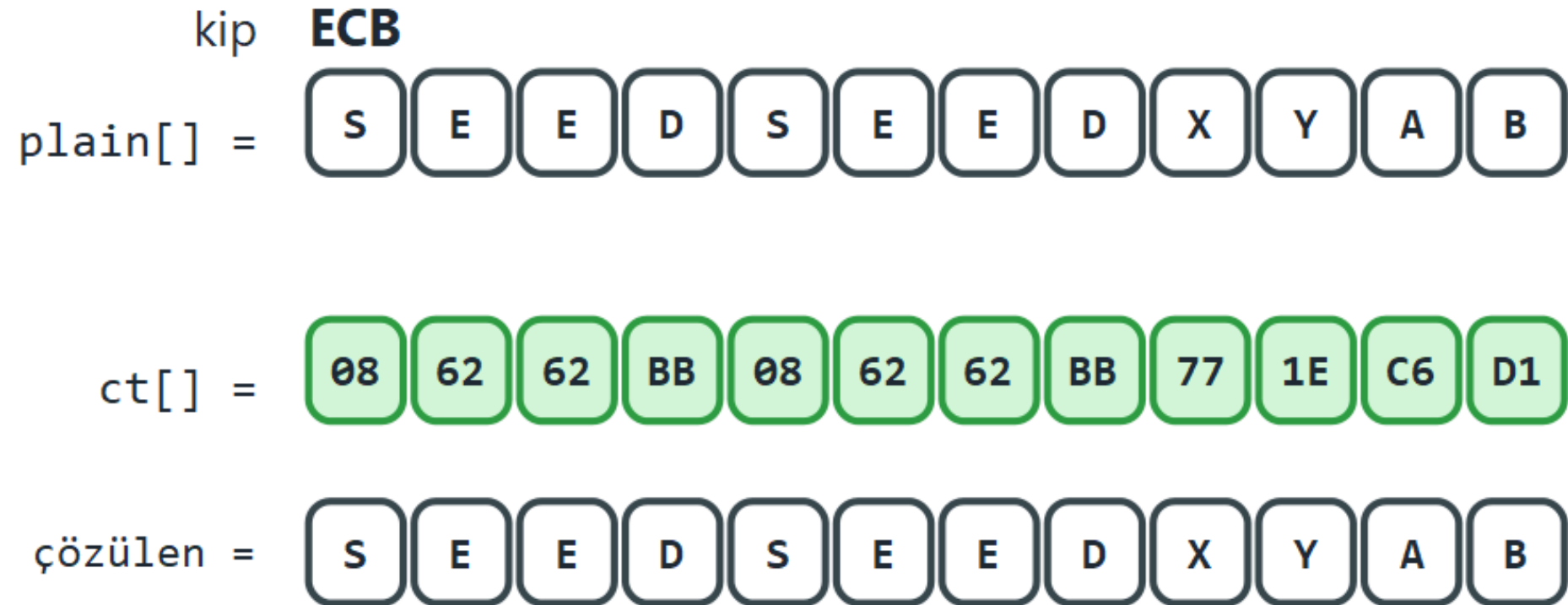
**Nonce gizli olmak zorunda değil ama anahtar başına BENZERSİZ olmak zorundadır.**

# ChaCha20-Poly1305

---

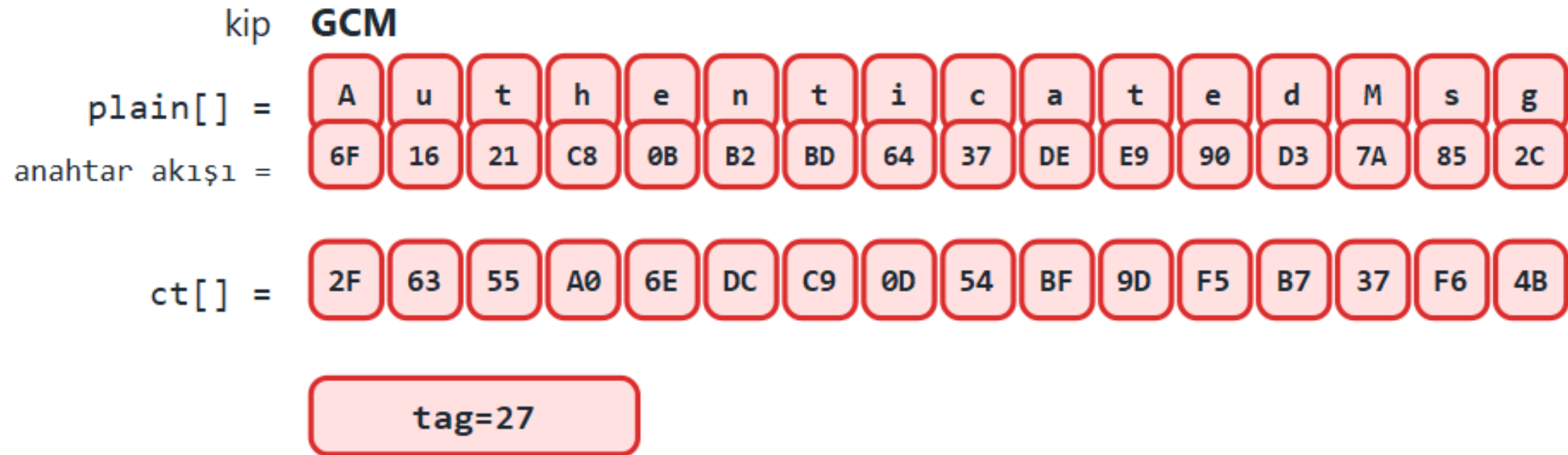
- AES-GCM'e AEAD alternatifi.
- Donanım AES desteği yoksa hızlı.
- Aynı nonce kuralı geçerli.

# Kipler karşılaştırması — animasyon



Çöz: her şey doğru çözüldü.

# Kipler · uç durum



REDDDEDİLDİ — hiçbir düz metin üretilmedi (fail-closed).

## Dolgu kâhini (padding oracle) · sorun adım adım

---

1. Sunucu, CBC çözerken **dolgu geçersiz** ve **MAC geçersiz** için **farklı** yanıt/zaman verir.
2. Saldırgan şifreli metni değiştirip yanıtlara bakar.
3. Yanıt farkından **düz metni** bayt bayt çözer.

## Neden oluyor?

---

- Hata mesajı/zamanı, iç durumu **sızdırıyor**.
- Saldırgan bunu bir **kâhin** (oracle) gibi kullanıyor.

## Çözüm

---

- **AEAD kullan** (GCM): dolgu yok, tek doğrulama.
- CBC zorundaysan: **encrypt-then-MAC** + **tek ve aynı** hata.
- Zamanlama farkı da sızıntıdır → sabit zamanlı.

# Ders

---

Şifreleme tek başına yetmez; **bütünlük** ve **tutarlı hata** şart.

Dolgu kâhini, "kullanım hatası"nın klasik örneğidir.

## Bölüm 1–2 — kısa sinama

---

1. En zayıf halka kuralını bir örnekle açıklayın.
2. ECB neden kullanılmaz?
3. GCM'de nonce tekrarı neden felakettir?
4. Dolgu kâhinini AEAD nasıl kapatır?

## Bölüm 1–2 — cevaplar

---

1. Sistem **en zayıf bileşeni** kadar güvenli. Örn. AES-256 kullanıp anahtarı düz dosyada tutmak → anahtar en zayıf halka.
2. **ECB** aynı açık bloğu aynı şifreli bloğa çevirir → desen sızar, anlamsal güvenlik yok.
3. Aynı anahtar+nonce **anahtar akışını** tekrarlar (gizlilik çöker) ve GHASH **kimlik anahtarı** ele geçirilir → felaket.
4. **AEAD** önce etiketi doğrular, yanlış etiketli metni **çözmeden reddeder** → dolgu hatası için **oracle** kalmaz.

## 3. MAC, HMAC ve bütünlük

# HMAC neden — şema

## Neden HMAC, neden $H(K||m)$ değil?

kendi MAC'inizi kurmayın

$H(K||m)$   
Merkle–Damgård özetlerde  
UZUNLUK UZATMA saldırısı  
saldırgan K'yı bilmeden  
 $H(K||m||\text{ek})$  üretebilir

**HMAC**  
iç içe iki özetleme  
ipad / opad ile  
uzunluk uzatma KAPALI  
standart ve kanıtlı

**Doğrulamayı sabit zamanlı karşılaştırmayla yapın (CRYPTO\_memcmp).**

## Neden MAC?

---

- Şifreleme gizliliği verir, **bütünlüğü vermez**.
- Saldırgan şifreli metni değiştirebilir.
- MAC: mesaj **değişmedi** ve doğru taraftan geldi.

# HMAC

---

- Özet fonksiyonuna dayalı MAC ( HMAC-SHA256 ).
- Simetrik anahtar; iki taraf aynı anahtarı bilir.
- Doğrulama **sabit zamanlı** karşılaştırmayla.

## HMAC-SHA-256 · gerçek çıktı

Bir ödeme talimatının HMAC'ini hesaplayalım (32 baytlık anahtar, yalnız gösterim için sabit):

```
printf 'tutar=100;alici=TR00' | \  
openssl dgst -sha256 -mac HMAC -macopt hexkey:$ANAHTAR
```

```
SHA2-256(stdin)= a08fb115...c5f9a1
```

## HMAC-SHA-256 · çığ etkisi

Aynı anahtarla, yalnız tutarı değiştirelim ( 100 → 900 ):

```
SHA2-256(stdin)= 8a786eac...91a46be
```

Tek karakterlik değişiklik ( 1 → 9 ), etiketin **tamamen farklı** çıkmasına yol açtı — buna **çığ etkisi** (avalanche effect) denir. Saldırgan anahtarı bilmeden geçerli bir yeni etiket üretemez; alıcı uyuşmazlığı yakalar.

# HMAC · iç/dış özet — animasyon

anahtar = 

|   |   |   |   |
|---|---|---|---|
| k | e | y | 1 |
|---|---|---|---|

K' = 

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 6B | 65 | 79 | 31 | 00 | 00 | 00 | 00 |
|----|----|----|----|----|----|----|----|

K' xor ipad = 

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 5D | 53 | 4F | 07 | 36 | 36 | 36 | 36 |
|----|----|----|----|----|----|----|----|

K' xor opad = 

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 37 | 39 | 25 | 6D | 5C | 5C | 5C | 5C |
|----|----|----|----|----|----|----|----|

mesaj = 

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| t | r | a | n | s | f | e | r | 1 | 0 | 0 | t | o | a | c | c | o | u | n | t |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

iç = 

|          |
|----------|
| D618403D |
|----------|

dış (HMAC) = 

|          |
|----------|
| 09E9BA8B |
|----------|

outer = SHA256(K'xor\_opad || inner) — Dış özet: bu, HMAC etiketinin kendisi.

## Sabit zamanlı karşılaştırma

```
/* YANLIS: ilk farkta durur, sure farki sizdirir */  
if (memcmp(hesaplanan, gelen, 32) == 0) { /* kabul */ }  
  
/* DOGRU: her zaman 32 baytin tamamini gezer */  
if (CRYPTO_memcmp(hesaplanan, gelen, 32) == 0) { /* kabul */ }
```

**Kural:** MAC/imza/parola özeti gibi gizli değerleri **her zaman** sabit zamanlı karşılaştırın; `memcmp` / `==` bir zamanlama saldırısına açık kapı bırakır.

# Encrypt-then-MAC — şema

## Encrypt-then-MAC: doğru sıra

alıcı geçersiz metni hiç çözmez → dolgu kâhini kapanır



**Yanlış sıra (önce çöz, sonra doğrula) dolgu hatası sızdırır ve oracle açar.**

## Doğru sıra · encrypt-then-MAC

```
1. şifrele:  c = ENC(k1, m)
2. MAC'le:  t = MAC(k2, c)
3. gönder:  c || t
```

- Önce şifrele, **sonra** şifreli metni MAC'le.
- MAC geçmezse **çözmeye bile kalkma**.

## Yanlış sıralar

---

- **MAC-then-encrypt**: dolgu kâhinine açık.
- **Encrypt-and-MAC**: MAC düz metni sızdırabilir.
- Doğrusu: **encrypt-then-MAC** (ya da AEAD, zaten böyle yapar).

# Sıra neden önemli — animasyon

mesaj: "SecretPayload01" (15 bayt)

S e c r e t P a y l o a d 0 1

Şifrele-sonra-MAC:

D1 6B A7 72 64 6F 0C 0A E8 FB 8B F9 FA 19 87 AD 6D

MAC-sonra-şifrele:

D1 6B A7 72 64 6F 0C 0A E8 FB 8B F9 FA 19 87 3C EB D8 2F B4 33 70 57 6C

Dolgu geçerliydi; mac KONTROL EDİLDİ ve tutuyor: düz metin geri verildi.

## Yeniden oynatma (replay)

---

- Saldırgan geçerli bir mesajı **tekrar** gönderir.
- MAC geçerli (mesaj değişmedi) ama işlem **tekrarlanır**.
- Çözüm: **nonce**, zaman damgası, sayaç.

## AEAD hepsini yapar

---

- AES-GCM: şifreleme + bütünlük **birlikte**.
- "İlişkili veri" (AAD) ile başlık/bağlamı da doğrular.
- Modern tercih: ayrı MAC yerine AEAD.

## Bu bölümün kuralı · MAC/HMAC

---

- Bütünlüğü **HMAC** ile sağlayın; düz  $H(K\parallel m)$  değil (uzunluk uzatma riski).
- Şifreleme + MAC birleşiminde sıra **önce şifrele, sonra MAC (encrypt-then-MAC)**'tir.
- Etiket/imza karşılaştırması **sabit zamanlı** olmalı; erken çıkışlı fonksiyon kullanmayın.

## 4. Asimetrik: RSA ve ECC

# Simetrik ↔ asimetrik — şema

## Simetrik ve asimetrik: hangisi ne için?

pratikte HİBRİT: asimetrikle anahtar kur, simetrikle veri şifrele

### **SİMETRİK (AES)**

tek anahtar

**ÇOK HIZLI**

sorun: anahtarı nasıl paylaşırsın?

### **ASİMETRİK (RSA/EC)**

açık + gizli anahtar çifti

**YAVAŞ**

anahtar dağıtımı ve İMZA çözülür

**TLS tam olarak bunu yapar: ECDHE ile anahtar, AES-GCM ile veri.**

# OAEP / PSS — şema

## RSA dolgusu: hangisi nerede?

dolgu, algoritmanın güvenliğinin ayrılmaz parçasıdır

### OAEP

RSA ile ŞİFRELEME  
rastgeleleştirilmiş dolgu  
ham RSA asla kullanılmaz

### PSS

RSA ile İMZA  
rastgeleleştirilmiş dolgu  
PKCS#1 v1.5 yerine tercih

**Eliptik eğride karşılıkları: X25519 (anahtar değişimi), Ed25519 (imza).**

## RSA · iki iş

---

- **Şifreleme:** açık anahtarla şifrele, gizli anahtarla çöz.
- **İmza:** gizli anahtarla imzala, açık anahtarla doğrula.
- Büyük anahtarlar (2048+).

## RSA · dolgu şart

---

- Ham RSA **güvensizdir**.
- **Şifreleme için**: OAEP dolgusu.
- **İmza için**: PSS dolgusu.
- Eski PKCS#1 v1.5: mümkünse kaçın.

# Oyuncak RSA · uçtan uca — animasyon

$$p = 61$$

$$q = 53$$

$$n = p \cdot q = 3233$$

$$\phi = (p-1)(q-1) = 3120$$

$$e = 17$$

$$d = 2753$$

$$m = 65$$

$$c = 2790$$

$$\text{decrypt}(c) = 65$$

$$\text{sign}(m) = 588$$

$$\text{verify} = \text{true}$$

$\text{verify}(m, \text{sig}, \text{pub}): \text{sig}^e \bmod n = 65; == m(65)? \text{EVET} \rightarrow \text{GEÇERLİ.}$

## OAEP vs PSS

---

- **OAEP:** RSA **şifreleme** dolgusu.
- **PSS:** RSA **imza** dolgusu.
- Karıştırma: OAEP imza için değil, PSS şifreleme için değil.

## OAEP'in rastgeleliği · komut

Aynı mesajı iki kez OAEP ile şifreleyelim:

```
openssl pkeyutl -encrypt -pubin -inkey rsa_acik.pem -in kisa.txt -out c1.bin \  
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256  
openssl pkeyutl -encrypt -pubin -inkey rsa_acik.pem -in kisa.txt -out c2.bin \  
-pkeyopt rsa_padding_mode:oaep -pkeyopt rsa_oaep_md:sha256  
cmp c1.bin c2.bin && echo AYNI || echo FARKLI
```

## OAEP'in rastgeleliği · sonuç

```
c1.bin c2.bin differ: char 1, line 1  
FARKLI
```

Aynı anahtar, aynı düz metin, ama **farklı** şifreli metin — OAEP her şifrelemede yeni bir rastgele değer karıştırır. **Ham RSA** (dolgunsuz) deterministiktir (  $c = m^e \bmod n$  ); bu yüzden hiçbir zaman doğrudan şifreleme için kullanılmaz.

## ECC · neden?

---

- Aynı güvenlik, **daha küçük** anahtar (256-bit  $\approx$  RSA-3072).
- Daha hızlı, daha az yer.
- Mobil/gömülü için ideal.

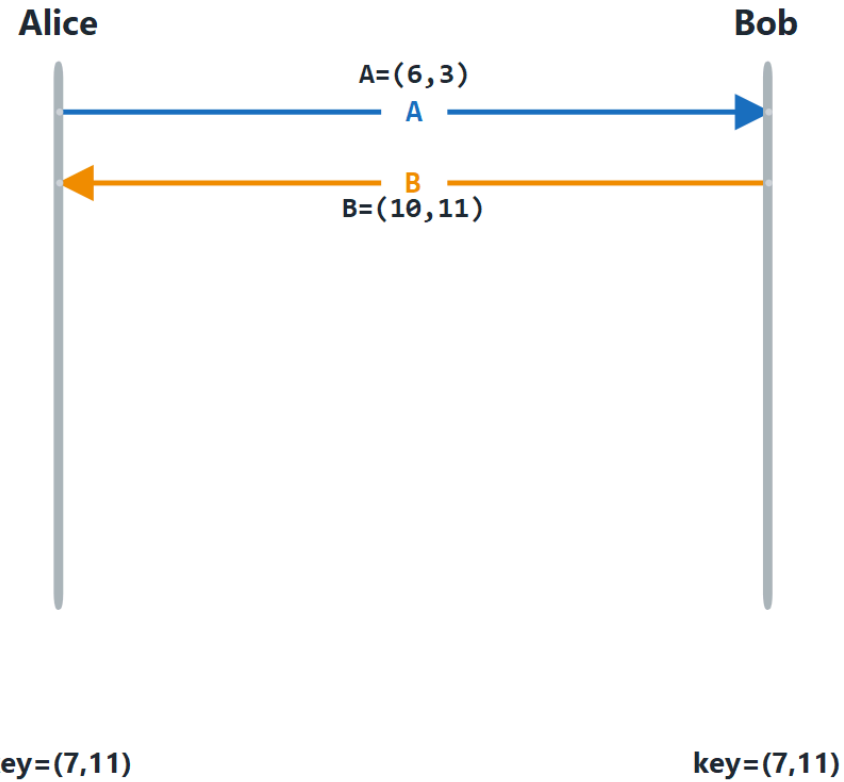
## Ed25519 ve X25519

---

- **Ed25519**: modern **imza** algoritması.
- **X25519**: modern **anahtar anlaşması** (DH).
- Karıştırma: Ed25519 imza, X25519 anahtar.

# ECDH · nokta toplama — animasyon

$$y^2 = x^3 + 2x + 2 \pmod{17}, G=(0,6)$$



Alice  $a \cdot B = (7, 11)$ , Bob  $b \cdot A = (7, 11)$  hesaplıyor; aynı nokta mı? EVET.

## RSA-3072 ile Ed25519 · ölçüm

---

Her ikisi de ~128 bit güvenlik düzeyinde (bkz. Bölüm 1 tablosu):

```
WC -c rsa_acik.pem ed_acik.pem  
WC -c belge.rsa.sig belge.ed.sig
```

## RSA-3072 ile Ed25519 · sonuç

---

| Dosya              | RSA-3072 | Ed25519  |
|--------------------|----------|----------|
| Açık anahtar (PEM) | 636 bayt | 116 bayt |
| İmza               | 384 bayt | 64 bayt  |

Açık anahtar **~5,5 kat**, imza **6 kat** küçük. IoT/mobil el sıkışmalarında, gömülü flash'ta ya da blok zincirinde bu fark birikerek büyür.

## Simetrik + asimetrik birlikte

1. X25519 ile ortak sır türet
2. Ondan bir AES anahtarı çıkar (KDF)
3. AES-GCM ile veriyi şifrele

- Asimetrik: anahtar taşı. Simetrik: veri şifrele.
- TLS bunu yapar.

## Bölüm 3–4 — kısa sinama

---

1. Encrypt-then-MAC neden doğru sıra?
2. Yeniden oynatma nasıl engellenir?
3. OAEP ve PSS hangisi ne için?
4. Ed25519 ve X25519 farkı?

## Bölüm 3–4 — cevaplar

---

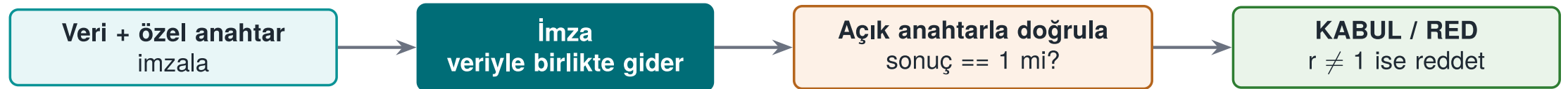
1. Alıcı **önce MAC'i** doğrular, geçmezse çözmez → değiştirilmiş metin hiç işlenmez (dolgu kâhini kapanır).
2. **Tazelik**: nonce/sayaç, zaman damgası + pencere, tek-kullanımlık challenge.
3. **OAEP** = RSA **şifreleme** dolgusu; **PSS** = RSA **imza** dolgusu.
4. **Ed25519** = imza (EdDSA); **X25519** = anahtar değişimi (ECDH). Aynı eğri ailesi, farklı iş.

## 5. Dijital imza ve tuzakları

# Dijital imza — şema

## Dijital imza: oluşturma ve doğrulama

en sık hata: if(r) ile denetlemek



**API 1=başarı, 0=başarısız, <0=hata döner; if(r) hatayı da “doğru” sayar.**

# İmza · ne sağlar?

---

- **Bütünlük:** mesaj değişmedi.
- **Kimlik/inkâr edilemezlik:** gizli anahtar sahibi imzaladı.
- **Doğrulama:** açık anahtarla, herkes yapabilir.

## İmza · nerede kullanılır?

---

- Yazılım/güncelleme imzalama.
- Sertifikalar (CA imzası).
- Belge/işlem imzalama.
- TLS'te sunucu kimliği (CertificateVerify).

## Tuzak 1 · dönüş değerini denetlememek

```
int r = EVP_DigestVerify(ctx, imza, n, veri, m);  
if (r) guncellemeyi_kur(); /* HATALI */
```

Negatif hata değeri de "doğru" sayılır. Doğrusu: `r == 1`.

## Tuzak 2 · sürümü imzaya katmamak

---

- İmza yalnız içeriği kapsıyorsa, saldırgan **eski** ama geçerli imzalı sürümü yükleyebilir (downgrade).
- İmzalı içerik **sürüm numarasını** da kapsamalı; eski sürüm reddedilmeli.

## Tuzak 3 · yanlış anahtarla doğrulama

---

- Doğrulama anahtarı güvenilir mi? Nereden geldi?
- Saldırganın anahtarıyla doğrularsanız, onun imzası "geçerli" olur.
- Anahtar **sabitlenmeli** ya da güvenilir zincirden gelmeli.

# İmza doğrulama · üç senaryo — animasyon

belge = U p d a t e P a c k a g e 0 1

imza=1287

**doğrula(signer.pub) = true**

openssl dgst -verify signer.pub -signature ... — sonuç: GEÇERLİ.

# İmza · özet kuralı

---

- İmza aslında **özetin** imzalanmasıdır.
- Zayıf özet (MD5/SHA-1) → imza zayıf.
- SHA-256+ kullan.

## 6. Diffie–Hellman ve MITM

## DH · ne yapar?

---

- İki taraf, gizli anahtar **paylaşmadan** ortak bir sır türetir.
- Ağ üzerinden anahtar anlaşması.
- Modern hali: X25519.

## DH · adım adım (kavram)

Ali:  $a$  gizli,  $A = g^a$  açık gönderir  
Veli:  $b$  gizli,  $B = g^b$  açık gönderir  
Ortak sır:  $A^b = B^a = g^{(ab)}$

Dinleyen  $g^a$ ,  $g^b$  görür ama  $g^{(ab)}$  'yi hesaplayamaz.

## X25519 · iki tarafın aynı sırra ulaşması

```
openssl genpkey -algorithm X25519 -out alice.key
openssl genpkey -algorithm X25519 -out bob.key
openssl pkey -in alice.key -pubout -out alice.pub
openssl pkey -in bob.key -pubout -out bob.pub

openssl pkeyutl -derive -inkey alice.key -peerkey bob.pub -out alice_sir.bin
openssl pkeyutl -derive -inkey bob.key -peerkey alice.pub -out bob_sir.bin
```

## X25519 · sonuç

```
8c53744a000c1a6f...bbc1376  
8c53744a000c1a6f...bbc1376  
AYNI
```

Alice ve Bob, birbirinin **hiç görmediği** özel anahtarını bilmeden, ağdan yalnız açık anahtarları geçirerek **aynı** 32 baytlık sırra ulaştı. Bu ham sır doğrudan AES anahtarı olarak kullanılmaz; önce bir **KDF**'den (HKDF) geçirilip oturum anahtarları türetilir.

## Kimliksiz DH · MITM riski

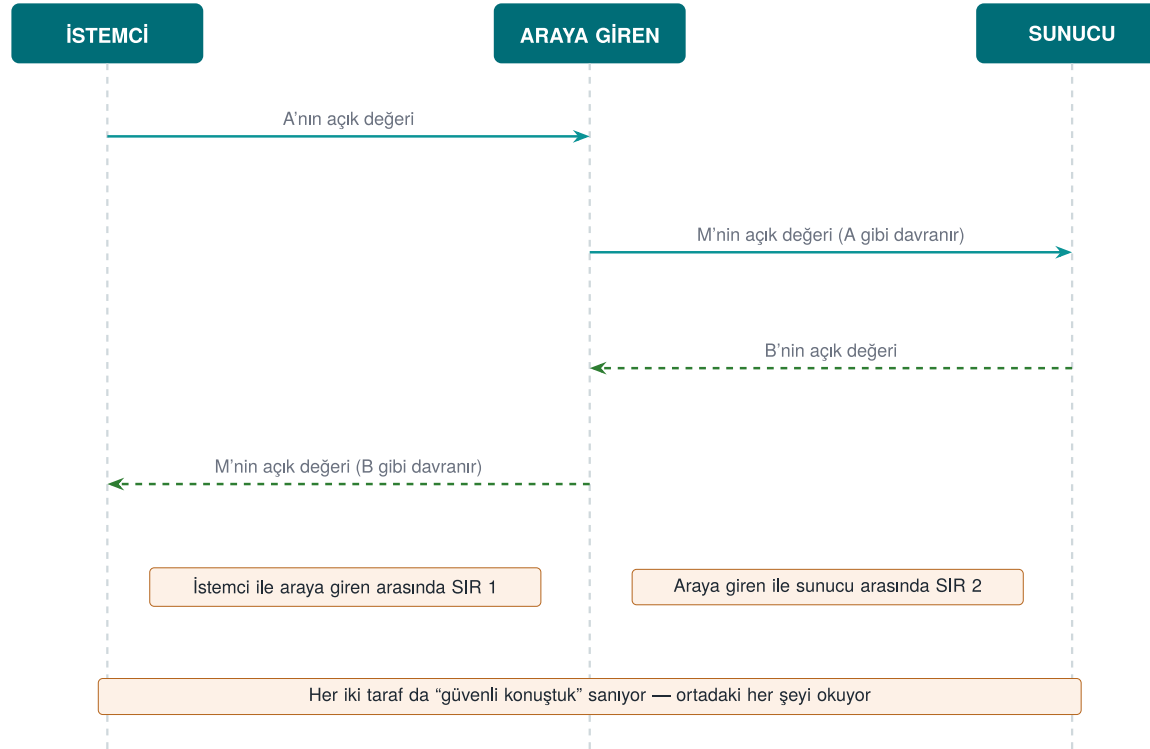
---

- Ali ve Veli birbirinin kim olduğunu **doğrulamıyorsa...**
- Saldırgan **araya girer**: Ali ile ayrı, Veli ile ayrı DH yapar.
- İkisini de dinler/değiştirir.

# Kimliksiz DH · MITM — şema

## Kimliksiz Diffie–Hellman: araya girme

DH kiminle anlaştığınızı SÖYLEMEZ



Çözüm: DH'yi kimlik kanıtıyla bağla (sertifika / imzalı DH) — TLS 1.3 bunu yapar.

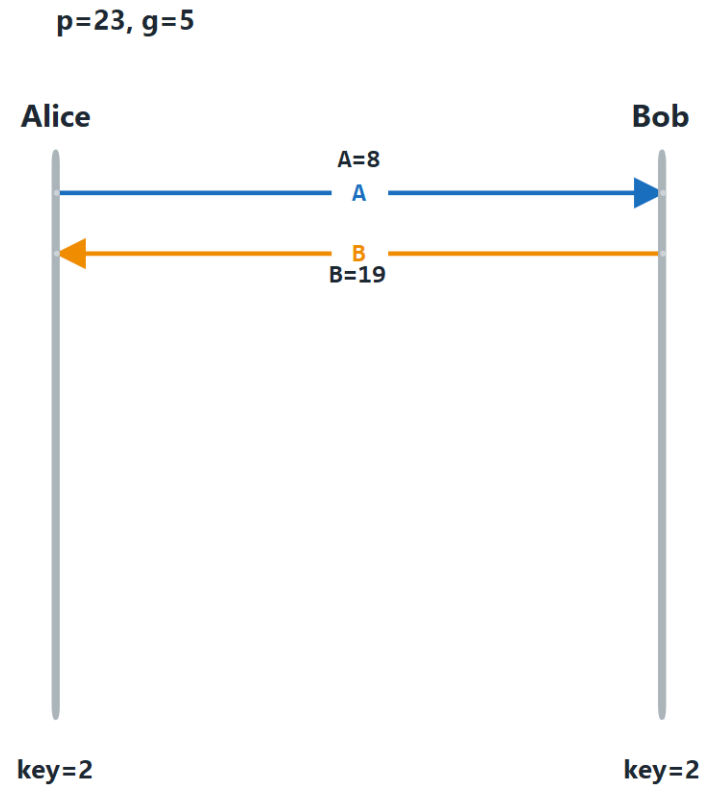
## MITM · şema

---

Ali ↔ [Saldırgan] ↔ Veli  
iki ayrı DH; saldırgan ortada

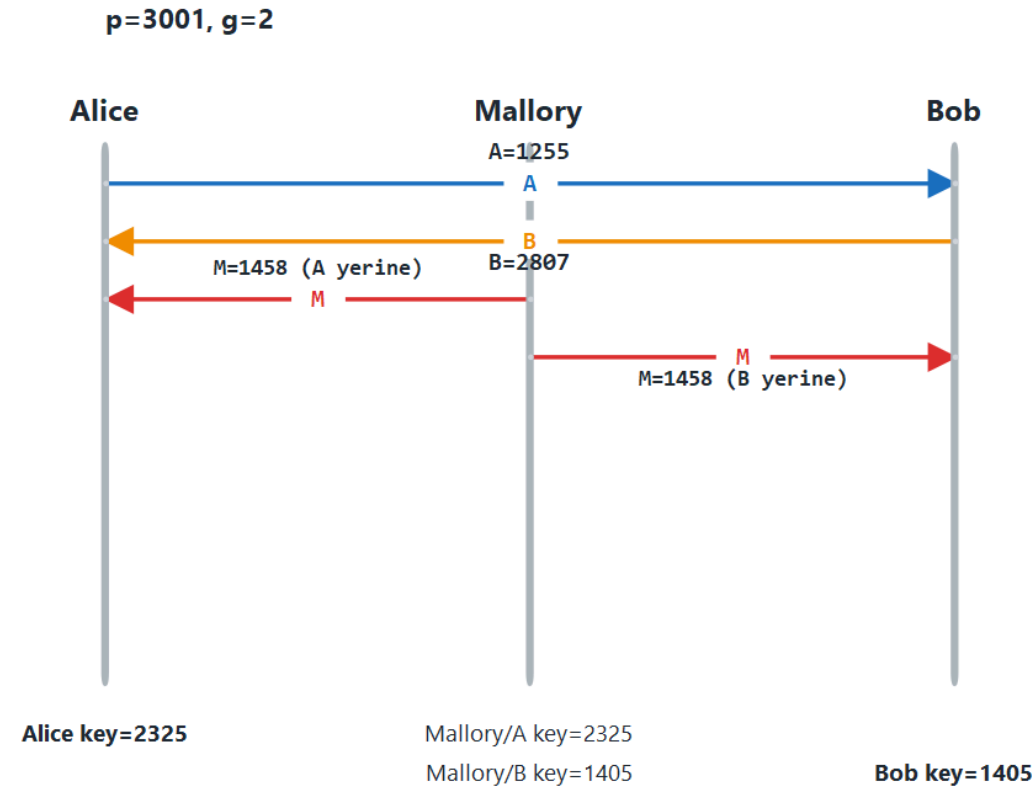
Her iki taraf "güvenli kanal kurdum" sanır.

# DH ve MITM — animasyon



shared\_secret(...) — Alice  $B^a \bmod p = 2$ , Bob  $A^b \bmod p = 2$ ; eşit mi? EVET.

# DH/MITM · uç durum



Alice anahtarı (2325) ile Bob anahtarı (1405) DOĞRUDAN eşit mi? HAYIR — ikisi de aslında Mallory ile konuşuyor.

## Çözüm · kimlikli DH

---

- DH değerleri, kimliği bilinen bir anahtarla **imzalanır**.
- TLS 1.3'te `CertificateVerify` .
- Karşı taraf bu kimliği **doğrular** (sertifika zinciri).

## İleri gizlilik (forward secrecy)

---

- Her oturum için **yeni** DH anahtarı.
- Uzun vadeli anahtar sızsa bile **eski** oturumlar çözülemez.
- Modern TLS bunu sağlar.

## Bölüm 5–6 — kısa sinama

---

1. İmza doğrulamada `if (r)` neden hatalı?
2. Sürüm imzaya neden katılmalı?
3. Kimliksiz DH'ye MITM nasıl olur? Çözüm?
4. İleri gizlilik ne sağlar?

## Bölüm 5–6 — cevaplar

---

1. Doğrulama 1=başarı, 0=başarısız, <0=hata döndürür; `if (r)` hatayı (–1) da "doğru" sayar. `r == 1` kontrol edin.
2. Aksi halde saldırgan **eski/güvensiz sürümü** aynı imzayla "geçerli" gösterir (downgrade). Sürüm imzalı veriye dahil olmalı.
3. Ortadaki her iki tarafla ayrı anahtar kurar. Çözüm: DH'yi **kimlik doğrulamayla** bağla (imzalı DH / sertifika).
4. Her oturum **geçici (ephemeral)** anahtar; uzun vadeli anahtar sızsa bile **geçmiş oturumlar** çözülemez.

## 7. Açık anahtar altyapısı (PKI)

## Sorun · kime güveneceğiz?

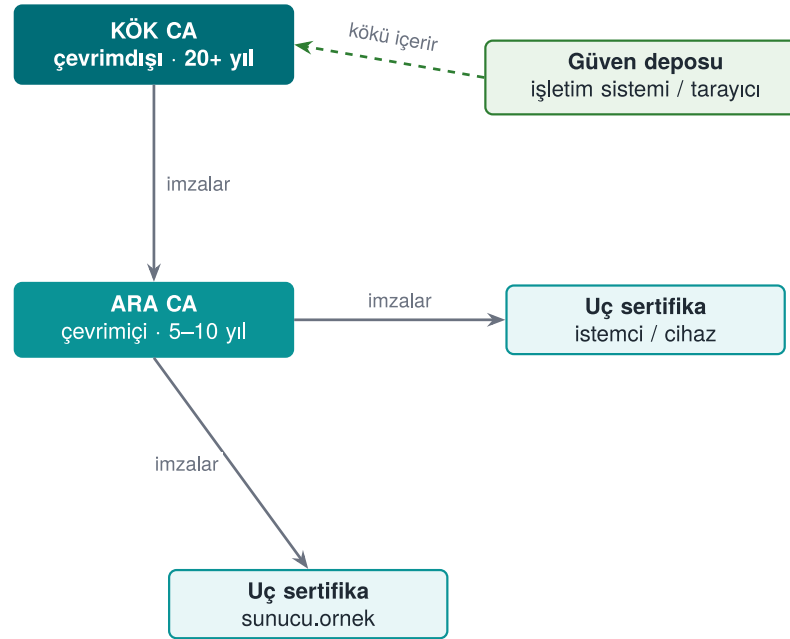
---

- Bir açık anahtar gördünüz. Gerçekten o kişiye/siteye mi ait?
- Kötü niyetli biri kendi anahtarını "banka" diye sunabilir.
- PKI bu güven sorununu çözer.

# Güven zinciri

## PKI: kim kime güvenir?

güven kökten uca akar



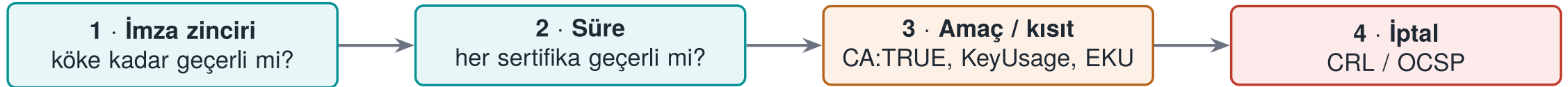
İstemci yalnız KÖKE güvenir; zincir kökten uca kesintisiz kurulamazsa doğrulama başarısızdır.

Her seviye bir alttakini **imzalar**.

# Zincir doğrulama — şema

## Zincir doğrulamanın dört sorusu

+ ad denetimi (SAN) — beşi birden geçmeden kabul edilmez



Eksik ara sertifika zinciri kurdurmaz; fail-open ile “geçti” saymak en sık hatadır.

## Kök ve ara CA

---

- **Kök CA:** çevrimdışı, kendini imzalar, çok korunur.
- **Ara CA:** günlük sertifikaları çıkarır.
- Kök sızarsa **felaket**; bu yüzden ara CA kullanılır.

## Güven deposu (trust store)

---

- İşletim sistemi/tarayıcı, güvenilen **kök CA'ları** taşır.
- Bir zincir bu köklerden birine ulaşırsa güvenilebilir.
- Ulaşamıyorsa → güvenilemez.

## Dört soru

---

Bir sertifika zinciri doğrularken:

1. **İmza** geçerli mi? (her seviye)
2. **Zincir** güvenilir köke ulaşıyor mu?
3. **Süre** dolmamış mı?
4. **Ad** eşleşiyor mu (SAN)?

Dördü de **evet** olmalı.

## Soru 1 · zincir (issuer/subject)

```
openssl x509 -in sunucu.crt -noout -issuer -subject  
openssl x509 -in ara.crt -noout -issuer -subject  
openssl x509 -in kok.crt -noout -issuer -subject
```

```
issuer=CN=...Ara CA subject=CN=localhost  
issuer=CN=...Kok CA subject=CN=...Ara CA  
issuer=CN=...Kok CA subject=CN=...Kok CA
```

Zincir okunur: `sunucu` 'nun issuer'ı = `ara` 'nın subject'i; kök **kendi kendini** imzalar.

## Soru 2 · geçerlilik

```
openssl x509 -in sunucu.crt -noout -dates  
openssl x509 -in sunucu.crt -noout -checkend 0
```

```
notBefore=Sep 23 2026 GMT  
notAfter=Dec 22 2026 GMT  
Certificate will not expire
```

-checkend 0 : "şu an itibarıyla süresi doldu mu?" TLS istemcileri bunu her bağlantıda otomatik sorar.

## Soru 3 · kullanım (CA:TRUE/FALSE)

```
openssl x509 -in sunucu.crt -noout -ext basicConstraints,keyUsage  
openssl x509 -in ara.crt -noout -ext basicConstraints,keyUsage
```

```
sunucu.crt:  CA:FALSE          (baska sertifika imzalayamaz)  
ara.crt:     CA:TRUE, pathlen:0 (yalniz uc sertifika imzalayabilir)
```

## Soru 4 · ad (SAN)

```
X509v3 Subject Alternative Name:  
DNS:localhost, IP Address:127.0.0.1
```

İstemci bağlandığı adı bu listeye karşılaştırır — Subject 'teki CN 'e **değil**.

Bir istemci kütüphanesi bu dört soruyu **sizin yerinize** sorar; **verify** 'ı kapatmak ya da hataları yutmak, dört sorunun hiç sorulmaması anlamına gelir.

## OpenSSL ile · örnek

---

```
openssl verify -CAfile kok.crt \  
-untrusted ara.crt sunucu.crt
```

- `-CAfile` : güvenilen kök.
- `-untrusted` : ara sertifika(lar).

## SPKI sabitleme (pinning)

---

- Uygulama, beklenen sunucu anahtarının **özetini** gömer.
- Sahte ama "geçerli" sertifika bile kabul edilmez.
- **Yedek pin** şart (anahtar değişince kilitlenmemek için).

## Bu bölümün kuralı · zincir doğrulama

---

- Zincir doğrulamanın **dört sorusundan** (imza, süre, kullanım, ad) hiçbirini atlamayın.
- `verify` ad denetlemez — **SAN** denetimi ayrıca yapılmalı.
- Sunucu **ara sertifikayı** göndermezse, kriptografik olarak doğru zincir bile bağlanamaz.

## 8. X.509 sertifikası

# X.509 alanları — şema

## X.509 sertifikasında ne var?

doğrulama bu alanların HEPSİNİ denetler

**Konu (Subject) + SAN** — kime ait — ad denetimi SAN'a bakar, CN'e değil

**Açık anahtar (SPKI)** — pinning bunu sabitler

**Geçerlilik aralığı** — notBefore / notAfter

**Uzantılar** — basicConstraints (CA:TRUE), KeyUsage, EKU

**CA imzası** — zincirin bir üst halkası tarafından

**Yalnız “imza geçerli” demek yetmez: ad, süre, amaç ve iptal de denetlenir.**

# İçinde ne var?

---

- **Konu (subject):** kime ait (alan adı).
- **Açık anahtar.**
- **Geçerlilik:** başlangıç/bitiş.
- **Veren (issuer):** hangi CA imzaladı.
- **İmza.**

## SAN · ad denetimi

---

- **SAN (Subject Alternative Name):** sertifikanın geçerli olduğu **alan adları**.
- Ad denetimi **CN'e değil, SAN'a** yapılır.
- `example.com` için sertifika `baska.com` 'da geçerli olmamalı.

## Gerçek sertifika · üst alanlar

```
Version: 3 (0x2)
Serial Number: 35:e2:f6:8d:....:76:e7
Signature Algorithm: ecdsa-with-SHA256
Issuer: CN=CEN429 Lab Ara CA
Validity: Sep 23 2026 - Dec 22 2026
Subject: CN=localhost
Subject Public Key Info: 256 bit, NIST CURVE: P-256
```

Gerçek `openssl x509 -text` çıktısı; her alan "dört soru"dan (bir sonraki bölüm) birinin cevabını taşır.

## Gerçek sertifika · uzantılar

```
X509v3 Basic Constraints: CA:FALSE  
X509v3 Key Usage: critical, Digital Signature  
X509v3 Extended Key Usage: TLS Web Server Authentication  
X509v3 Subject Alternative Name:  
    DNS:localhost, IP Address:127.0.0.1  
X509v3 Authority Key Identifier: F5:BA:39:86:...
```

Authority Key Identifier, sertifikayı imzalayan CA'nın **parmak izidir**; zinciri kurarken "aynı isimli ama farklı anahtarlı sahte ara CA" karışıklığını bununla önler.

## 9. OpenSSL ile sertifika zinciri kurmak

# Amaç

---

Küçük bir zincir kuralım:

Kök CA → Ara CA → Sunucu sertifikası.

Sonra `verify` ile doğrulayalım ve tipik hatayı görelim.

## Adım 1 · kök CA (kendini imzalar)

---

```
openssl req -x509 -newkey ed25519 \  
-keyout kok.key -out kok.crt \  
-subj "/CN=Ders Kok CA" -days 3650 -nodes
```

Kök kendini imzalar; çevrimdışı tutulur.

## Adım 2 · ara CA (kök imzalar)

---

```
openssl req -newkey ed25519 -keyout ara.key \  
  -out ara.csr -subj "/CN=Ders Ara CA" -nodes  
openssl x509 -req -in ara.csr -CA kok.crt -CAkey kok.key \  
  -CAcreateserial -out ara.crt -days 1825
```

## Adım 3 · sunucu sertifikası (ara imzalar)

---

```
openssl req -newkey ed25519 -keyout sunucu.key \  
  -out sunucu.csr -subj "/CN=ornek.test" -nodes  
openssl x509 -req -in sunucu.csr -CA ara.crt -CAkey ara.key \  
  -CAcreateserial -out sunucu.crt -days 365
```

## Adım 4 · doğrula (eksik ara)

---

```
openssl verify -CAfile kok.crt sunucu.crt  
# HATA: unable to get local issuer certificate
```

Zincir köke ulaşamıyor: **ara sertifika eksik.**

## Adım 5 · doğrula (ara ile)

---

```
openssl verify -CAfile kok.crt \  
    -untrusted ara.crt sunucu.crt  
# OK
```

Ara sertifikayı verince zincir tamamlanır.

## Ders

---

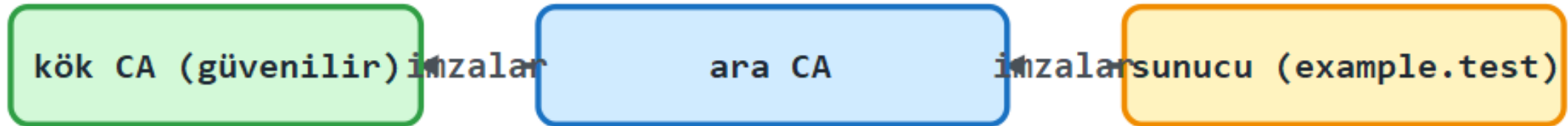
- Sunucu, **ara** sertifikayı da göndermeli.
- Eksikse istemci "issuer bulunamadı" der.
- Sahadaki en yaygın TLS yapılandırma hatası.

## Zincirdeki dört soru · uygulaması

---

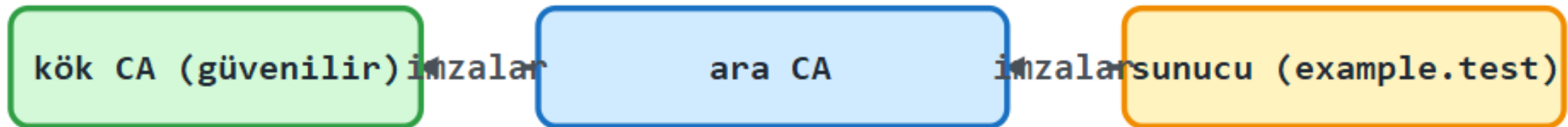
- **İmza:** her seviye üsttekince imzalı ✓
- **Kök:** `-CAfile kok.crt` güven deposu ✓
- **Süre:** `-days` içinde ✓
- **Ad:** uygulamada SAN denetimi (verify ad denetlemez!)

# Zincir doğrulama · altı senaryo — animasyon



`-verify_hostname example.test` — SAN eşleşiyor: ZİNCİR KABUL EDİLDİ.

# Zincir · uç durum (süresi dolmuş)



-atime — kontrol tarihinde sunucu sertifikası SÜRESİ DOLMUŞ.

## Sertifika doğrulamayı kapatma

---

```
/* ASLA: */  
SSL_CTX_set_verify(ctx, SSL_VERIFY_NONE, NULL);
```

Bu, MITM'e kapıyı açar. "Test için" diye bırakılan bu satır sahada felakettir.

## Ad denetimini atlamak

---

- Zincir geçerli ama **ad** doğrulanmıyorsa, saldırganın geçerli sertifikası kabul edilir.
- Ana bilgisayar adı (SAN) mutlaka denetlenir.

## Eleştirel okuma kuralı

---

Bir TLS kurulum kodunu okurken sorun:

- Doğrulama açık mı?
- Ad denetleniyor mu?
- Hata durumunda **fail-closed** mı?

# 10. Sertifika iptali

# Sertifika iptali — şema

## Sertifika iptali: CRL · OCSP · zımbalama

“bu sertifika hâlâ geçerli mi?”



**FAIL-OPEN TUZAĞI: “erişilemedi” = “geçti” sayma!**

## Neden iptal?

---

- Bir sertifikanın gizli anahtarı sızarsa, süresi dolmadan **iptal** gerekir.
- İptal edilmiş sertifika kabul edilmemeli.

# CRL

---

- **CRL:** iptal edilmiş sertifikaların **periyodik listesi**.
- Büyük olabilir, güncel olmayabilir.

# OCSP

---

- **OCSP:** bir sertifikanın durumunu **anlık** sorma.
- Daha güncel; ama gizlilik/hız sorunları.

## OCSP yanıtını iliştiirme (stapling)

---

- Sunucu, OCSP yanıtını **kendisi** getirip sertifikayla sunar.
- Gizlilik + hız kazanır.

## CRL iş akışı · iptal et ve üret

```
openssl ca -config ara.cnf -revoke sunucu.crt  
openssl ca -config ara.cnf -gencrl -out ara.crl
```

```
Revoking Certificate 35E2F68D...76E7.  
Database updated
```

Sertifika dosyası **değişmez**; yalnız CA'nın defterine ( `index.txt` ) bir iptal kaydı eklenir. `-gencrl` , bu kaydı CA'nın kendi anahtarıyla **imzalı** bir listeye dönüştürür.

## CRL ile doğrulama · sonuç

```
openssl verify -crl_check -CAfile kok.crt \  
-untrusted ara.crt -CRLfile ara.crl sunucu.crt
```

```
error 23 at 0 depth lookup: certificate revoked  
error sunucu.crt: verification failed
```

Sertifikanın **süresi hâlâ dolmamış** olsa bile CRL'de listelendiği için reddedildi — iptal denetimi, süre denetiminden **bağımsız** çalışır.

# CRL vs OCSP — animasyon

**sertifika, seri #4711**

**yayımlandı, iptal edilmedi**

**OCSP:**

`ocsp_query(#4711) = "good"`

**CRL:**

`crl_check(#4711) = "good"`

`crl_check(4711, crl, at_time)` — CRL hâlâ taze, sonuç: "good".

## Fail-open tuzağı

---

- OCSP'ye ulaşamayınca sertifikayı **kabul etmek** = fail-open.
- Saldırgan OCSP'yi engelleyip iptal edilmiş sertifikayı geçirebilir.
- Azaltma: **Must-Staple**, kısa ömürlü sertifikalar.

## Bölüm 7–10 — kısa sinama

---

1. Zincir doğrulamanın dört sorusu?
2. Ad denetimi neye yapılır (CN mi SAN mı)?
3. Eksik ara sertifika neden `verify` 'ı bozar?
4. Fail-open nedir, nasıl azaltılır?

## Bölüm 7–10 — cevaplar

---

1. (1) zincir kök'e **geçerli** mi, (2) **süre** içinde mi, (3) **amaç/kısıt** (CA:TRUE, KeyUsage/EKU), (4) **iptal** (CRL/OCSP).
2. **SAN** (Subject Alternative Name); CN artık kullanılmaz.
3. Zincir kök'e **kurulamaz**; güven yolu tamamlanmaz → başarısız. Sunucu **ara** sertifikayı da göndermeli.
4. **Fail-open**: hata/erişilemezlikte "geçti" saymak. Azaltma: **fail-closed**, OCSP stapling / must-staple, hatayı reddet.

# 11. Anahtarı donanımda saklamak

## HSM nedir?

---

- **HSM:** anahtarları saklayan/işleten özel donanım.
- Anahtar HSM'in **dışına çıkmaz**; "şunu imzala" dersiniz, sonuç döner.
- Kurcalamaya dirençli.

## PKCS#11

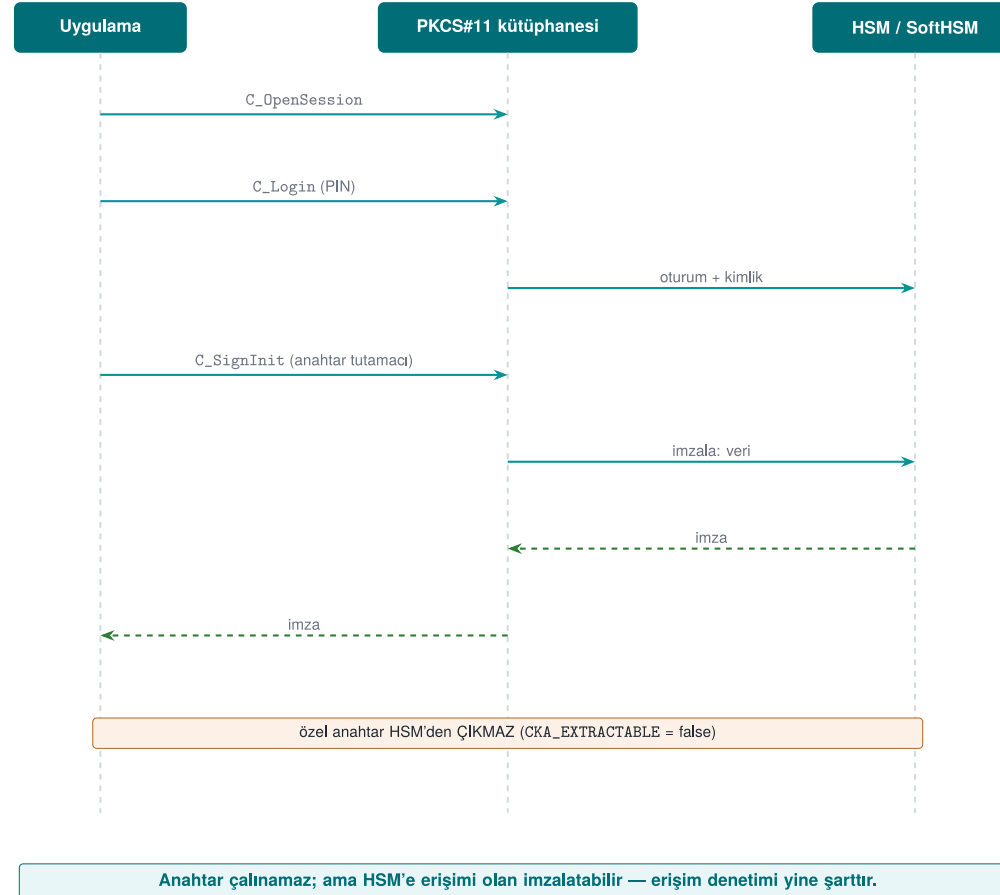
---

- Anahtar modülleriyle konuşmanın **standart arayüzü**.
- Uygulama anahtarın değerini **görmez**.
- HSM'ler ve yazılım benzetimleri bu arayüzü sunar.

# PKCS#11 ile imzalama — şema

## PKCS#11 ile imzalama: anahtar hiç dışarı çıkmaz

uygulama anahtar değil, anahtarın TUTAMACINI görür



# Anahtar hiç çıkmaz — animasyon

HSM (anahtar HİÇ çıkmaz)

kulp #1 üretildi

belge =

P u r c h a s e o r d e r 0 1

imza=36976267

**verify(1, belge, sig) = true**

verify(1, belge, sig) — aynı kulp: imza doğrulanıyor.

# SoftHSM

---

- HSM'in **yazılım benzetimi**; aynı PKCS#11 arayüzü.
- Geliştirme/test için; **gerçek donanım koruması yok**.
- Üretimde gerçek HSM.

## Neden donanım?

---

- Yazılımda anahtar, beyaz kutu saldırgana açık (11. hafta).
- Donanımda anahtar yalıtılmış → çok daha güçlü.
- Mümkünse anahtarı **yazılıma hiç koyma**.

# Anahtar yaşam döngüsü

| Aşama          | Ne yapılır                 |
|----------------|----------------------------|
| Üretim         | Güçlü rastgelelik (CSPRNG) |
| Saklama        | HSM/TEE ya da korunmuş     |
| Kullanım       | En az yetki, sabit zamanlı |
| Kripto-periyot | Ömür sınırı                |
| Yenileme       | Düzenli                    |
| İmha           | Bellekten güvenli silme    |

## Bu bölümün kuralı · anahtar saklama

---

- Değerli bir anahtar üretilirken `CKA_EXTRACTABLE=false` **açıkça** ayarlanmalı; varsayılanına güvenmeyin.
- HSM'de anahtar hiç çıkmaz; uygulama yalnız bir **tutamaç** (handle) üzerinden "imzala" der.
- SoftHSM test içindir; üretimde gerçek HSM/donanım koruması gerekir.

## 12. Kuantum sonrası kriptoloji

# Kuantum sonrası — şema

## Kuantum sonrası: neyi değiştiriyor, neyi değiştirmiyor?

NIST seçimi: ML-KEM (Kyber) ve ML-DSA (Dilithium)

### TEHDİT ALTINDA

RSA, DH, ECDH, ECDSA

“şimdi topla, sonra çöz”

bugünkü trafik ileride çözülebilir  
geçiş bugün planlanır

### BÜYÜK ÖLÇÜDE GÜVENLİ

AES-256 (anahtar boyu yeter)

SHA-384 / SHA-3

simetrik dünya ayakta  
yalnız anahtar boyu artar

**Uzun ömürlü sırlarınız varsa hibrit anahtar değişimine geçiş bugünün kararıdır.**

# Tehdit

---

- Yeterince güçlü kuantum bilgisayar, bugünkü **RSA/ECC**'yi kırabilir.
- "Şimdi topla, sonra çöz" saldırısı: bugün şifreli veriyi sakla, gelecekte çöz.

## PQC · yaklaşım

---

- Kuantuma dayanıklı algoritmalar (ör. ML-KEM anahtar kapsülleme).
- Standartlaşma sürüyor.
- **Kripto-çeviklik:** algoritmayı kolay değiştirebilecek tasarım.

## PQC boyutları · komut

---

OpenSSL 3.5, standartlaşan PQC algoritmalarını **bugün** üretebilir:

```
openssl genpkey -algorithm ML-KEM-768 -out mlkem.pem
openssl genpkey -algorithm ML-DSA-65 -out mldsa.pem
openssl pkeyutl -sign -inkey mldsa.pem -rawin -in belge.txt -out belge.mldsa.sig
wc -c mlkem_pub.pem mldsa_pub.pem belge.mldsa.sig
```

## PQC boyutları · sonuç

| Algoritma         | Açık anahtar | İmza       |
|-------------------|--------------|------------|
| Ed25519 (klasik)  | 116 bayt     | 64 bayt    |
| RSA-3072 (klasik) | 636 bayt     | 384 bayt   |
| ML-KEM-768 (PQC)  | 1.714 bayt   | —          |
| ML-DSA-65 (PQC)   | 2.770 bayt   | 3.309 bayt |

ML-DSA-65: açık anahtar Ed25519'dan **~24 kat**, imza **~52 kat** büyük. TLS'te **karma** (klasik + PQC) yaklaşım bu geçiş maliyetini yönetmek içindir.

## Bugün ne yapmalı?

---

- Kripto-çevik ol (algoritmayı sabitleme).
- Uzun ömürlü sırlar için PQC'yi izle.
- Panik yok; ama hazırlık.

## 13. Proje: bu hafta (S8, S11)

## Proje · S8/S11

---

- [ ] **Algoritma envanteri:** amaç, algoritma, kip, anahtar uzunluğu, kütüphane.
- [ ] **Anahtar yaşam döngüsü** tablosu.
- [ ] TLS/sertifika doğrulaması; varsa sabitleme + yedek pin.
- [ ] İmza doğrulaması (güncelleme dosyasında).

# Uçtan uca: bir kriptto tasarım vakası

---

Bir mobil uygulama:

- Yerel hassas veriyi şifreliyor.
- Sunucuyla güvenli konuşuyor.
- Güncellemeleri doğruluyor.

Kriptto tasarımını yapalım.

## Yerel veri şifreleme

---

- **AES-256-GCM** (AEAD): gizlilik + bütünlük.
- Nonce sayaç/rastgele, **tekrarsız**.
- Anahtar TEE/HSM'de; yoksa whitebox (11).

# Sunucu iletişimi

---

- **TLS 1.3:** X25519 anahtar anlaşması + AEAD.
- Sertifika zinciri doğrulama + (varsa) SPKI sabitleme + yedek pin.
- Fail-open değil.

# Güncelleme doğrulama

---

- **Ed25519** imza; `verify == 1`.
- İmzalı içerik **sürümü** kapsar; downgrade reddi.
- Doğrulama anahtarı sabit/gömülü (bütünlükle korunur).

## Anahtar envanteri (S8)

| Anahtar         | Amaç            | Alg/uzunluk   | Saklama           |
|-----------------|-----------------|---------------|-------------------|
| Veri anahtarı   | Yerel şifreleme | AES-256       | TEE/whitebox      |
| Oturum anahtarı | TLS             | X25519 türevi | Bellek, kısa ömür |
| İmza doğrulama  | Güncelleme      | Ed25519 açık  | Gömülü            |

## Klasik krypto hataları — özet

| Hata                        | Bölüm | Doğrusu                                      |
|-----------------------------|-------|----------------------------------------------|
| Kendi kriptonu yazmak       | 1     | Denenmiş kütüphane, standart algoritma       |
| ECB kipi                    | 2     | AEAD (GCM / ChaCha20-Poly1305)               |
| IV/nonce tekrarı            | 2     | Benzersiz, tekrarsız nonce                   |
| Şifreleme var, bütünlük yok | 3     | AEAD ya da encrypt-then-MAC                  |
| Ham RSA / yanlış dolgu      | 4     | OAEP (şifre), PSS (imza)                     |
| Zayıf özet                  | 5     | SHA-256+                                     |
| İmza dönüşünü denetlememek  | 5     | <code>== 1</code> ve sürüm kapsanır          |
| Zincir/ad denetlememek      | 7     | Dört soru + SAN denetimi                     |
| Fail-open iptal             | 10    | Must-Staple, kısa ömür                       |
| Anahtarı düz gömmek         | 11    | HSM/TEE; yoksa whitebox + katman (11. hafta) |

On hatanın hepsi bu destede işlendi; burada tek bakışta toparlıyoruz.

# Çözümlü kendini sinama

# Soru 1

---

**RSA-2048 + AES-256 sisteminin güvenlik düzeyi?**

**Cevap:** ~112 bit (en zayıf halka RSA-2048). 128 bit için RSA-3072 ya da ECC.

## Soru 2

---

"Dolgu geçersiz" ve "MAC geçersiz" ayrı iletiler: risk?

**Cevap:** Dolgu kâhini; saldırgan yanıtlara bakıp mesajı çözebilir. Tek ve aynı hata; tercihen AEAD.

## Soru 3

---

`if (EVP_DigestVerify(...))` neden hatalı?

**Cevap:** Negatif hata değeri de doğru sayılır; denetim `== 1` olmalı. Ayrıca imzalı içerik sürümü kapsamalı.

## Soru 4

---

**verify** ara sertifika olmadan neden başarısız?

**Cevap:** Zincir köke ulaşamaz; ara sertifika eksik. Sahada sunucu ara sertifikayı göndermiyordur.

## Soru 5

---

`openssl verify` komutuna hem doğru hem alakasız bir ara sertifika verildiğinde ne olur?

**Cevap:** OpenSSL zinciri kurabildiği herhangi bir geçerli yoldan kurar (ör. doğrudan köke); alakasız sertifika yalnızca kullanılmayan bir aday olarak kalır, doğrulama yine de başarılı olabilir. Bu kriptografik değil operasyonel bir durumdur.

## Soru 6

---

**ECDSA'da aynı rastgele değerin iki imzada kullanılması neden felakettir?**

**Cevap:** İki imzadan özel anahtar hesaplanabilir. Ed25519 ya da deterministik ECDSA (RFC 6979) bu riski ortadan kaldırır.

## Soru 7

---

**Kök CA neden çevrimdışı tutulur ve ara CA neden vardır?**

**Cevap:** Kök anahtarın ele geçirilmesi bütün zinciri çökertir ve kökü güven depolarından çıkarmak yıllar sürer.

Günlük işi ara CA yapar; ele geçirilirse yalnız o iptal edilir.

## Soru 8

---

**Anahtarı neden yazılıma koymamalı? Alternatif?**

**Cevap:** Beyaz kutu saldırgan yazılımdaki anahtarı çıkarabilir. Alternatif: HSM/TEE (PKCS#11); yoksa whitebox + katman (11. hafta).

## Sözlük

| Terim            | Anlam                         |
|------------------|-------------------------------|
| AEAD             | Şifreleme + bütünlük birlikte |
| Encrypt-then-MAC | Doğru sıra                    |
| OAEP/PSS         | RSA şifre/imza dolgusu        |
| Ed25519/X25519   | İmza / anahtar anlaşması      |
| SAN              | Sertifika ad denetimi         |
| CRL/OCSP         | İptal listesi / anlık sorgu   |

## Sözlük (ek)

| Terim           | Anlam                          |
|-----------------|--------------------------------|
| Nonce           | Tek kullanımlık sayı           |
| KDF             | Anahtar türetme fonksiyonu     |
| İleri gizlilik  | Eski oturumlar sonra çözülemez |
| Trust store     | Güvenilen kök CA'lar           |
| Kripto-periyot  | Anahtar ömrü                   |
| Kripto-çeviklik | Algoritmayı kolay değiştirme   |

# Bir sonraki hafta

## 11. hafta — Whitebox kriptografi

Bu hafta anahtarı korumanın "donanıma emanet et" yolunu (HSM, PKCS#11, `CKA_EXTRACTABLE=false`, Bölüm 11) gördük.

11. hafta aynı soruyu donanım yokken sorar: bir anahtar, hiçbir HSM'e erişimi olmayan, tamamen yazılımdan ibaret bir uygulamanın içinde nasıl korunur? Tablo tabanlı whitebox AES ve ona karşı geliştirilen saldırılar (BGE, DCA, DFA), bu haftaki "anahtar hiç açığa çıkmasın" ilkesinin yazılım tarafındaki karşılığıdır.

Bu haftanın özeti: Kriptoda güvenlik doğru algoritmadan çok doğru **kullanımdadır** — AEAD, doğru dolgu, doğrulanan imza, denetlenen zincir, iyi yönetilen anahtar. Kriptoyu "açık/kapalı" değil, doğru kullanılıp kullanılmadığıyla değerlendirin.